Universidade Federal Rural do Semi-Árido – UFERSA
Departamento de Computação – DC

Funções de Hash e Autenticação de Mensagens

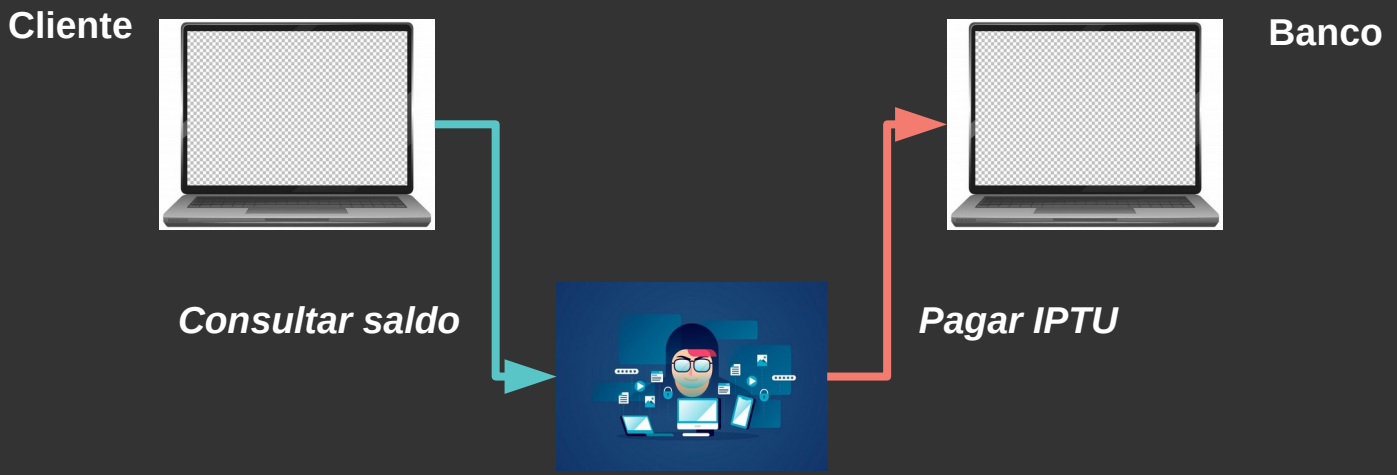
Segurança Computacional

Prof. Paulo Henrique Lopes Silva

Roteiro

- Funções de hash.
- Códigos de autenticação.

Relembrando integridade



Integridade com redundância

- Exemplo não criptográfico (simplificado):
  - Dígito verificador (DV): usado em RG/CPF, bancos, etc.
    - Um método: **`mod 11`**.
      - Cada dígito ocupa uma posição, alocando um peso para cada uma.
        - Da direita para a esquerda começando pelo peso 2.
      - Somam-se os resultados.
      - DV: é o resto da divisão da soma por 11.

Entrada	3	4	6	8	3
Posição	6	5	4	3	2
Multiplicação	18	20	24	24	6
Soma	92				
DV	92 mod 11 = 4				

Função de Hash

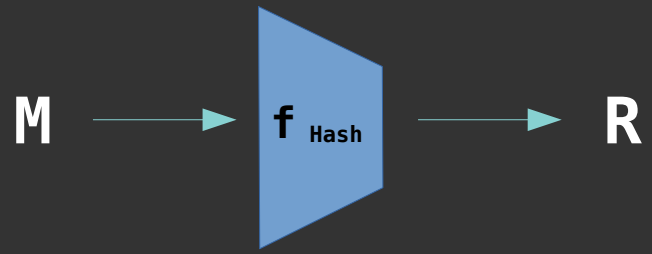
- Recebe, como entrada, uma mensagem e produz, como saída, um código hash de tamanho fixo.
  - Em função de todos os bits da mensagem e oferece a possibilidade de detectar erros.
    - Uma mudança em qualquer bit da mensagem, resulta em mudança no código hash.
  - Independe de qualquer informação secreta (chave).
  - Entrada: sequência de blocos de n bits.
    - Um bloco, por vez, é processado para produzir um hash ou resumo criptográfico.

Função de Hash

Mensagem de
comprimento variável

$$f_{Hash}(M) = R\ (MD)$$

Um valor de
comprimento fixo
MD: *Message Digest*



Função de Hash

- Uma função hash produz uma impressão digital de um arquivo, mensagem ou outro bloco de dados.
- Sobre a função, é necessário que:
  - Possa ser aplicada a um bloco de dados de qualquer tamanho.
  - Produzir uma saída de tamanho fixo.
  - Ser relativamente fácil calcular, $f_{Hash}(X) = R$, para qualquer entrada **X**.

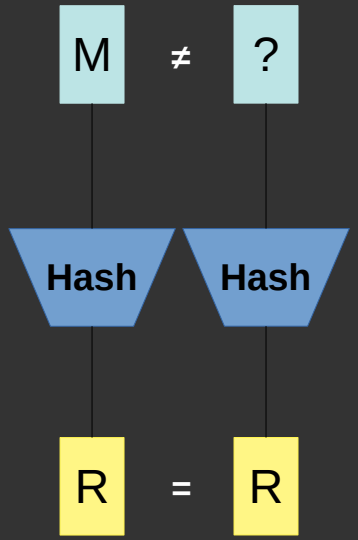
Função de Hash: propriedades

Resistência a 1ª inversão



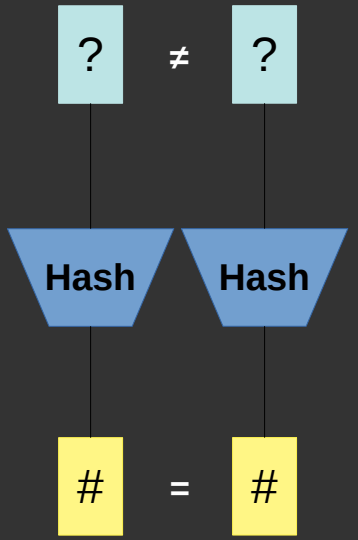
Esforço de 2^n

Resistência a 2a inversão



Esforço de 2^n

Resistência a colisões



Esforço de $2^{n/2}$

Função de Hash: propriedades

- Para qualquer valor **R** dado, deve ser computacionalmente inviável encontrar **X** tal que $f_{Hash}(X) = R$.
  - Resistência a primeira inversão ou propriedade unidirecional *(one-way property)*.
  - A função de hash deve se comportar como uma "via de mão única": fácil calcular o hash de um dado, mas impraticável recuperar o dado a partir do hash.
  - Exemplo prático: Um atacante que intercepta apenas o hash de uma senha não deve conseguir derivar a senha original (a menos que tente ataques de força bruta ou dicionário, que ainda assim seriam inviáveis se a senha for forte e o hash tiver *salting* adequado).

Função de Hash: propriedades

- Considere um bloco de dados **X** e seu código hash.
  - É computacionalmente inviável obter um bloco de dados **Y** diferente que gere o mesmo hash (resistência a segunda inversão).
  - Conhecer um dado válido e seu hash não deve permitir "forjar" outro dado diferente com o mesmo hash.
  - Exemplo prático: Se alguém assina digitalmente o hash de um documento X, não deve ser possível criar outro documento com o mesmo hash para enganar a verificação da assinatura.

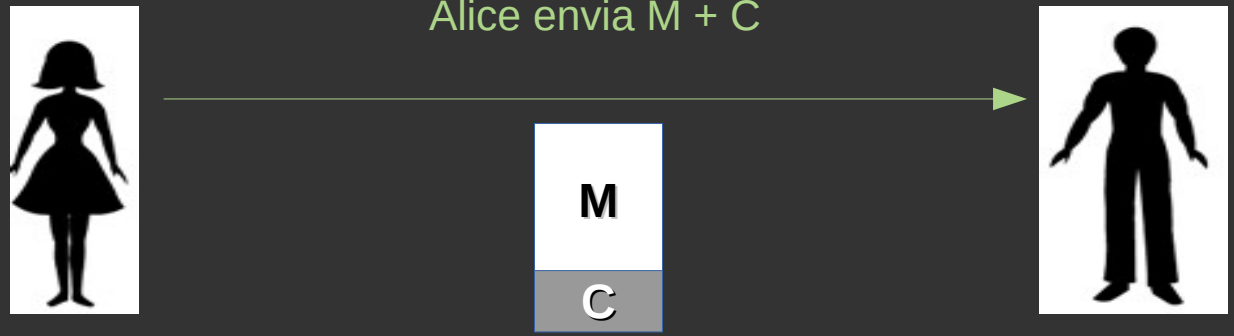
Função de Hash: propriedades

- Para qualquer bloco de dados **X**, é computacionalmente inviável encontrar **Y ≠ X**, tal que $f_{Hash}(Y) = f_{Hash}(X)$.
  - Resistência fraca a colisões.
  - Diferença em relação à segunda inversão:
    - Aqui, não se parte de um hash R dado nem de um objetivo específico.
    - O problema é: dado X, não é viável achar qualquer outro Y com o mesmo hash.
  - Exemplo prático: Em sistemas de verificação de integridade de arquivos, mesmo conhecendo o arquivo original X, o atacante não deve conseguir criar outro arquivo com o mesmo hash e enganar o sistema.

Função de Hash: propriedades

- Resistência forte a colisões *(collision resistance)*.
  - Deve ser computacionalmente inviável encontrar qualquer par de mensagens diferentes **(X,Y)** tal que: $f_{Hash}(X) = f_{Hash}(Y)$ com $X \neq Y$.
  - Diferença em relação à resistência fraca: Aqui não há restrição de começar por um dado específico.
    - O atacante só precisa encontrar qualquer colisão.
    - Isso é ainda mais difícil de garantir.
  - Exemplo prático: Em um ataque teórico, se alguém conseguir encontrar dois documentos diferentes com o mesmo hash SHA-1, isso quebra a resistência forte a colisões (o que de fato ocorreu com o SHA-1 em 2017, no ataque ***SHAttered***).

Função de Hash: cenário



$F_{Hash}(M) = R$
$F_{K_{AB}}(R) = C$

$F^{-1}_{K_{AB}}(C) = R$
$F_{Hash}(M) = R'$

se (R = R')
Integridade OK
Autenticidade* OK

*Considerando que nenhum outro usuário conhece K_{AB}

Função de Hash: algoritmos

- Família MD *(Message Digest)*: MD2 (1989), MD4 (1990) e MD5 (1991).
  - Hashes de 128 bits (16 bytes): 32 caracteres hexadecimais.
  - MD5.
    - 1996: primeiras colisões teóricas descobertas.
    - 2004 (Wang et al.): ataques práticos de colisão viáveis foram demonstrados.
    - 2008: ataques de colisão usados para forjar certificados digitais falsos (um grande marco que mostrou o risco real).
    - Hoje: colisões podem ser geradas em segundos em hardware comum.

Função de Hash: algoritmos

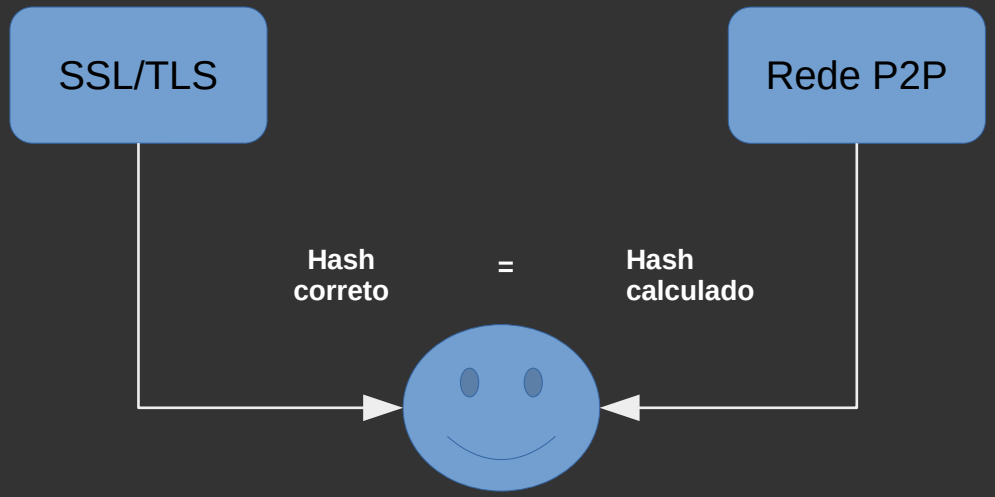
- Família SHA *(Secure Hash Algorithm)*.
  - SHA-0 (1993): hashes de 160 bits (20 bytes).
    - Baseado no MD4/MD5 com mais rodadas.
    - Rapidamente substituído pelo SHA-1, porque continha uma falha de segurança (detectada internamente pela NSA, não documentada publicamente).
    - **Não recomendado**: colisão em 2^{39} passos (2^{80} no projeto).
  - SHA-1 (1995): hashes de 160 bits (20 bytes).
    - Corrigiu a falha do SHA-0.
    - Foi adotado como padrão mundial em SSL/TLS, PGP, SSH, IPsec e em certificados digitais X.509.
    - 2017: primeiro ataque prático de colisão, conhecido como SHAttered (feito pelo Google e CWI Amsterdam).
      - Exemplo: dois PDFs diferentes com o mesmo hash SHA-1.
    - **Não recomendado**: colisão em 2^{60} passos (2^{80} no projeto).

Função de Hash: algoritmos

- Família SHA *(Secure Hash Algorithm)*.
  - SHA-2 (2001): hashes de 224, 256, 384 ou 512 bits.
    - Baseado na estrutura de SHA-1, mas com mudanças significativas para maior segurança.
    - Existem duas famílias internas:
      - SHA-224 e SHA-256: usam blocos de 512 bits.
      - SHA-384 e SHA-512: usam blocos de 1024 bits.
    - Até hoje, não há colisões práticas conhecidas.
  - SHA-3 – Keccak (2015, com base em concurso de 2012): hashes de 224, 256, 384 ou 512 bits.
    - Estrutura totalmente diferente dos SHA anteriores (não é baseado em MD4/MD5 como os demais).
    - Flexível: pode ser usado não só para hash, mas também para autenticação (KMAC), geração de números aleatórios e criptografia autenticada.
    - Recomendado como alternativa e sucessor de SHA-2.

Função de Hash: exemplo

- Download de programas via web, P2P, etc.
  - Hash deve ser obtido por canal confiável.
    - Se hash calculado ≠ hash correto: descarte.



Aplicações

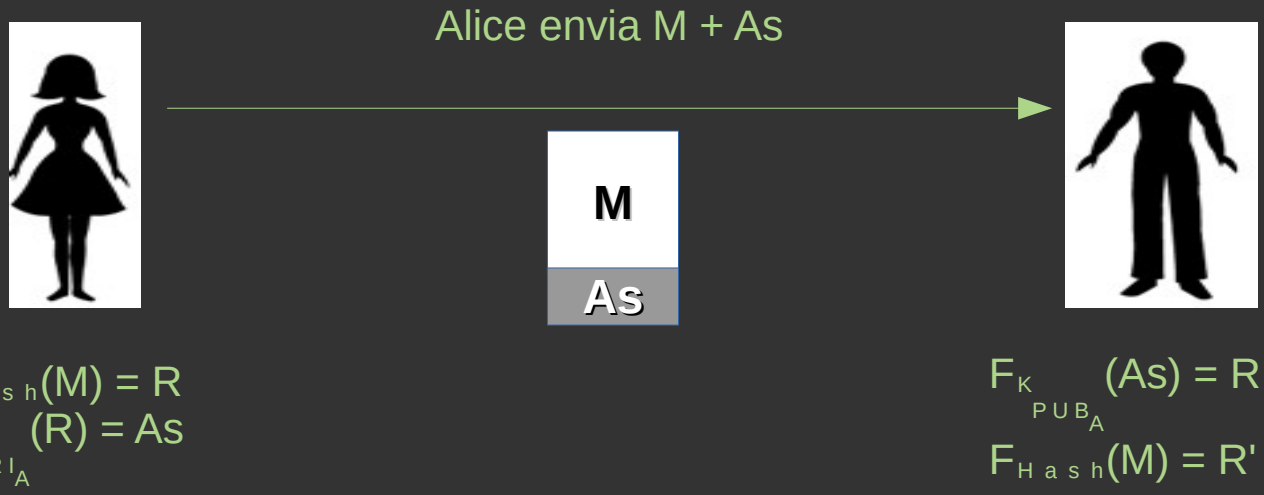
- Armazenamento de senhas.
  - Guardar o hash das senhas é mais seguro na ocorrência de ataques ao servidor da aplicação.
  - Torna-se mais complicado obter as senhas de todos os usuários dessa aplicação.

Aplicações

- Assinatura digital.
  - Criptografar um documento com um algoritmo assimétrico (chave pública e privada) usando a chave privada do emissor.
    - Algoritmos assimétricos são mais lentos.
    - Documento pode ser grande, tornando esse processo computacionalmente custoso.
  - Solução: gerar hash (tamanho fixo) do documento e criptografá-lo.
    - Receptor calcula o hash usando a mensagem recebida e compara com o hash enviado.

Aplicações

- Assinatura digital.



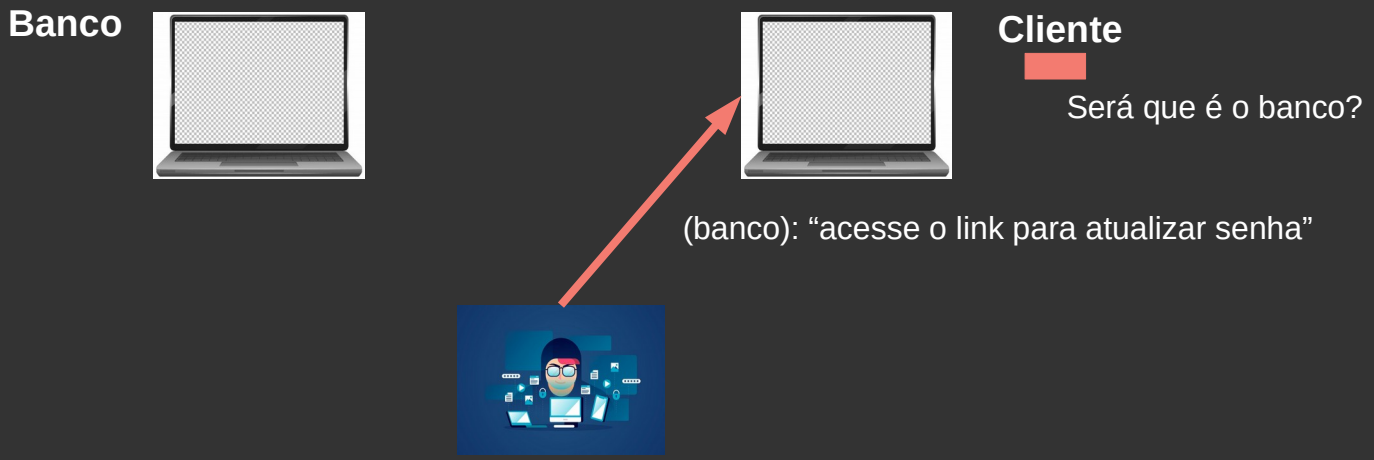
Aplicações

- Indexação de dados para busca.
  - Considere uma base de dados com milhões de imagens.
  - Fazer uma busca.
    - Comparar imagem por imagem está fora de questão.
  - Gravar na base de dados registros contendo a imagem e seu hash.
    - Assim, a busca se torna mais eficiente.

Códigos de Autenticação

Relembrando sobre autenticidade

- Receptor verificar quem é o emissor da mensagem.



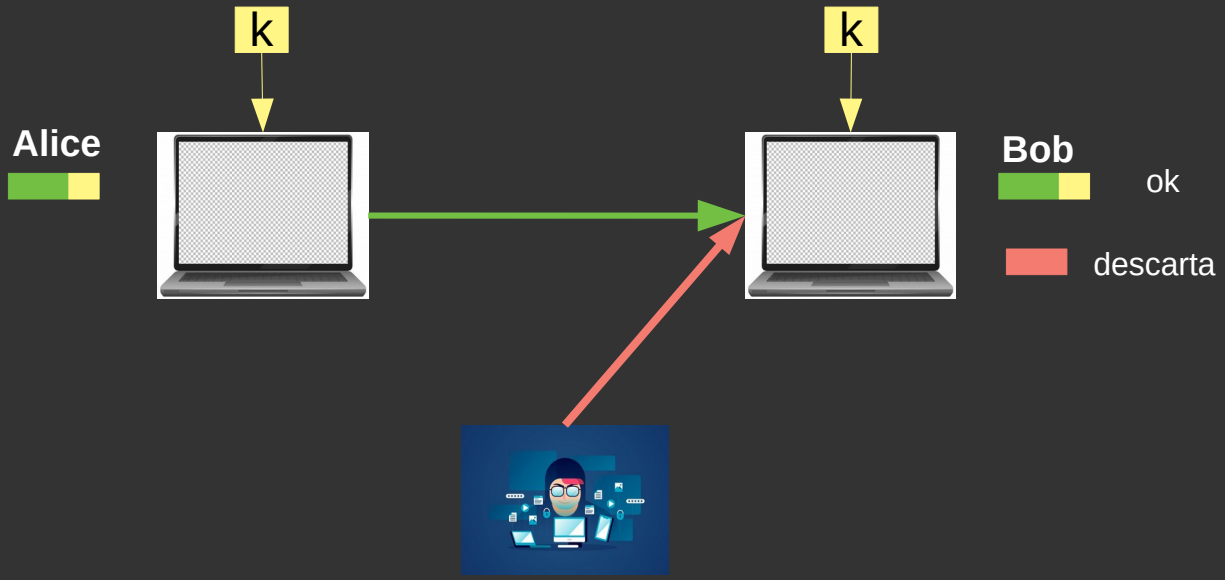
Autenticidade

- Hash pode ser usado?
  - Sozinho não funciona: qualquer um pode calcular o hash de uma mensagem.
  - Mensagem integra não significa que foi enviada pelo banco.

Autenticidade

- O que pode ser usado?
  - Redundância dependente de chave.
    - Apenas origem e destino tem a chave e conseguem calcular a tag de autenticação (redundância).
  - Integridade também é obtida.
    - Qualquer alteração na mensagem pode ser detectada (assim como nas funções de hash).
  - Em geral, usa-se hash com a adição da chave.

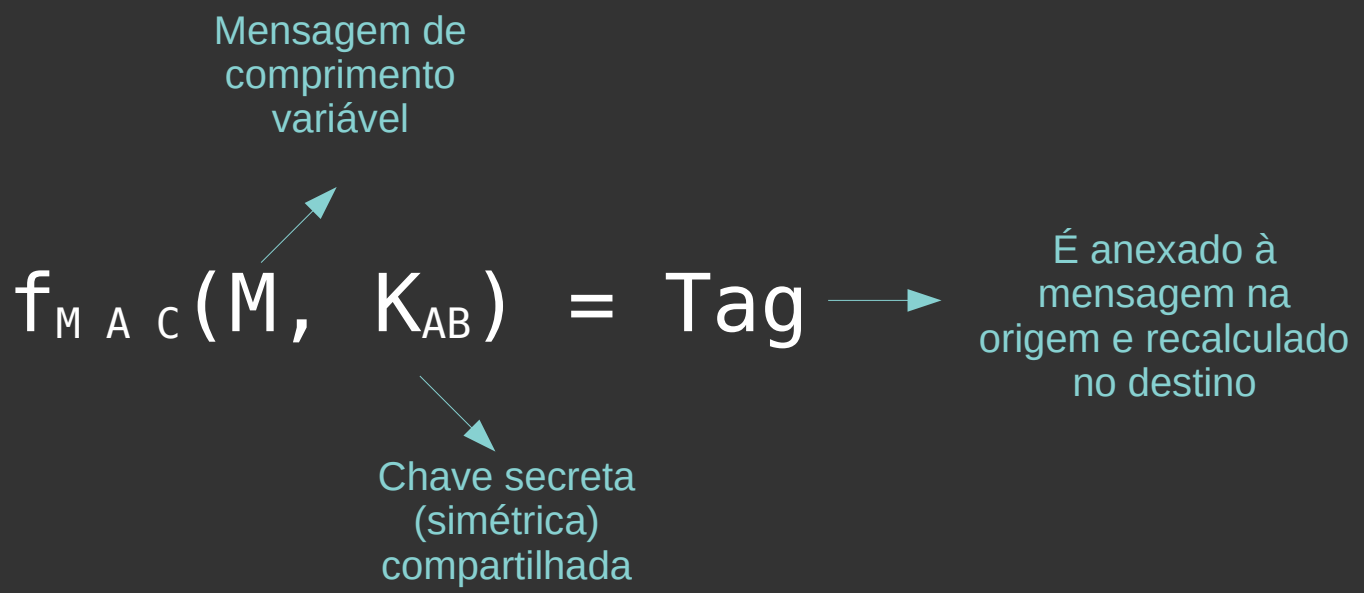
Autenticidade



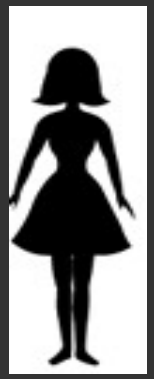
Códigos de autenticação

- Redundâncias anexadas a mensagens visando detectar alterações (integridade) e garantir autenticidade do emissor.
- Também chamados de MAC *(Message Authentication Code)*:
  - Um código (bloco de dados de tamanho fixo) é gerado em função da mensagem e de uma chave secreta (simétrica), que serve como autenticador.

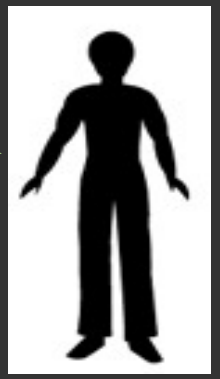
MAC e tag de autenticação



Cenário de uso



Alice envia M + Tag



M

Tag

$F_{MAC}(M, K_{AB}) = Tag$

$F_{MAC}(M, K_{AB}) = Tag'$

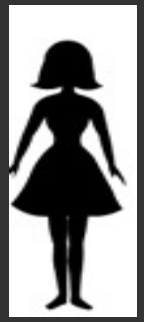
se (Tag = Tag')

Integridade OK

Autenticidade OK*

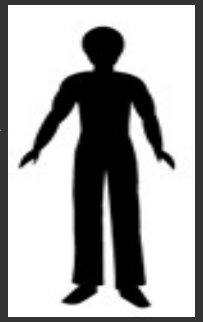
*Considerando que nenhum outro usuário conhece K_{AB}

Cenário de uso



Alice envia C

C



$F_{MAC}(M, K_{AB}) = Tag$

$F_{K_{AB}}(M + Tag) = C$

$F^{-1}_{K_{AB}}(C) = M + Tag$

$F_{MAC}(M, K_{AB}) = Tag'$

*Considerando que nenhum outro usuário conhece K_{AB}

se (Tag = Tag')

Integridade OK

Autenticidade OK*

Confidencialidade OK

MAC - Propriedades

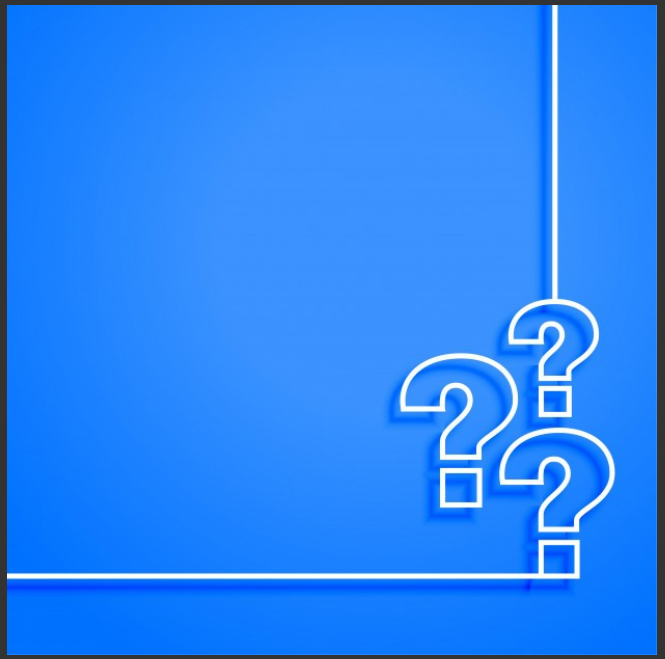
- Integridade:
  - Se alguém alterar a mensagem, o MAC gerado pelo receptor não irá coincidir.
  - Permite detectar alterações.
- Autenticidade:
  - Apenas quem conhece a chave secreta pode gerar/verificar o MAC.
  - Garante que a mensagem veio de quem diz ter enviado.
- Eficiência:
  - Mais rápido que assinaturas digitais (não usa chave pública/privada).
  - Usado em protocolos de rede (TLS, IPsec, etc.).

Algoritmos

- CMAC: baseado em cifras de bloco.
- HMAC: baseados em funções de hash.
  - Força depende da função de hash usada.
  - Bom desempenho.
- Combinados com cifras:
  - AEAD *(Authenticated Encryption with Associated Data).*

Dúvidas?

- Acessem os canais de comunicação da disciplina.



Referências

- W. Stallings. Criptografia e Segurança de Redes. Princípios e Práticas. Quarta Edição, Pearson, 2008.
- UNIVESP. Segurança da Informação - Aula 03 - Funções de Hash, códigos de autenticação e números aleatórios. 2018. (22m58s). Disponível em <https://www.youtube.com/watch?v=c5g51DXNV8o>. Acesso em: 15 set. 2026.